
Spring Security

ОСНОВЫ

Общая структура проекта

Проект демонстрирует основные концепции Spring Security:

- Аутентификация - проверка кто ты
- Авторизация - что тебе разрешено делать
- Хранение паролей - безопасное хранение учетных данных

1. Модель пользователя (User.java)

Класс реализует интерфейс UserDetails - это "контракт" Spring Security для работы с пользователями

Spring Security использует этот интерфейс для получения информации о пользователе.

Метод `getAuthorities()` возвращает список ролей с префиксом "ROLE_"

Важно понять:

- `ROLE_` - это стандартный префикс Spring Security для ролей
- В коде мы пишем `hasRole('ADMIN')`, но Spring Security ищет `ROLE_ADMIN`
- `GrantedAuthority` - это просто обертка над строковой ролью

2. Сервис пользователей (UserService.java)

а) Хранение в памяти

```
private final List<User> users = new ArrayList<>();
```

б) Реализация UserDetailsService

```
public class UserService implements UserDetailsService
```

- Spring Security требует этот интерфейс для поиска пользователей
- Метод loadUserByUsername() вызывается автоматически при аутентификации

2. Сервис пользователей (UserService.java)

с) Регистрация с хешированием

```
user.setPassword(passwordEncoder.encode(rawPassword));
```

- Пароль никогда не хранится в открытом виде
- Даже администратор не может узнать пароль пользователя

d) Проверка пароля

```
if (!passwordEncoder.matches(oldPassword, user.getPassword())) {  
    throw new RuntimeException("Old password is incorrect");  
}
```

- Нельзя просто сравнить строки, нужно использовать matches()
- matches() сравнивает введенный пароль с хешем

3. Конфигурация безопасности (SecurityConfig.java)

а) Отключение CSRF (для API)

```
http.csrf(csrf -> csrf.disable())
```

- CSRF защита важна для веб-форм, но не нужна для REST API
- API обычно используют токены (JWT, Basic Auth)

б) Настройка прав доступа

```
.authorizeHttpRequests(authz -> authz
    .requestMatchers("/api/auth/**", "/api/public/**").permitAll()
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN")
    .anyRequest().authenticated()
)
```

permitAll() - доступно всем (регистрация, логин)

hasRole("ADMIN") - только для админов

hasAnyRole("USER", "ADMIN") - для пользователей и админов

authenticated() - любой аутентифицированный пользователь

3. Конфигурация безопасности (SecurityConfig.java)

с) HTTP Basic Authentication

```
.httpBasic(httpBasic -> httpBasic.realmName("Demo App"))
```

- Basic Auth передает username:password в заголовке
- Кодировка Base64 (не шифрование!)
- Всегда используйте HTTPS в реальных проектах

4. Кодировщики паролей (PasswordEncoder)

BCrypt (используем в проекте):

```
@Bean  
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

Характеристики BCrypt:

- Автоматически добавляет **соль** (random salt)
- Адаптивный алгоритм (можно менять сложность)
- Одинаковые пароли дают разные хеши
- Устойчив к атакам по радужным таблицам

```
String hash1 = passwordEncoder.encode("password");  
String hash2 = passwordEncoder.encode("password");  
// hash1 != hash2, но оба валидны
```


4. Кодировщики паролей (PasswordEncoder)

Алгоритм	Когда использовать	Особенности
BCrypt	Современные приложения	Рекомендуемый, адаптивный
Argon2	Высокая безопасность	Победитель конкурса Password Hashing
PBKDF2	Legacy системы	Используется в Android, iOS
SCrypt	Защита от ASIC-атак	Ресурсоемкий
NoOpPasswordEncoder	ТОЛЬКО ДЛЯ ТЕСТОВ	Не шифрует пароли!

5. Контроллер (AuthController.java)

Аннотации авторизации:

```
@PreAuthorize("isAuthenticated()")
```

```
@PreAuthorize("hasRole('ADMIN')")
```

```
@PreAuthorize("hasAnyRole('USER', 'ADMIN')")
```

Получение текущего пользователя:

```
Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
```

Важно: SecurityContextHolder хранит информацию в ThreadLocal (привязан к текущему потоку)

6. Способы аутентификации

Метод	Описание	Пример использования
Form Login	HTML форма	Веб-приложения с UI
JWT	JSON Web Token	Масштабируемые API
OAuth2	Социальные сети, SSO	Интеграция с Google, GitHub
LDAP	Корпоративные системы	Active Directory
API Key	Простые API	Сервис-ту-сервис

7. Важные концепции безопасности

Принципы хранения паролей:

1. Никогда не храните в открытом виде
2. Всегда используйте соль (salt)
3. Используйте адаптивные алгоритмы (BCrypt, Argon2)
4. Регулярно обновляйте алгоритмы

Принципы авторизации:

1. Принцип наименьших привилегий - давать минимум прав
2. Defense in depth - несколько уровней защиты
3. Не доверяйте пользовательскому вводу
4. Всегда проверяйте права на сервере